LABSHEET 4

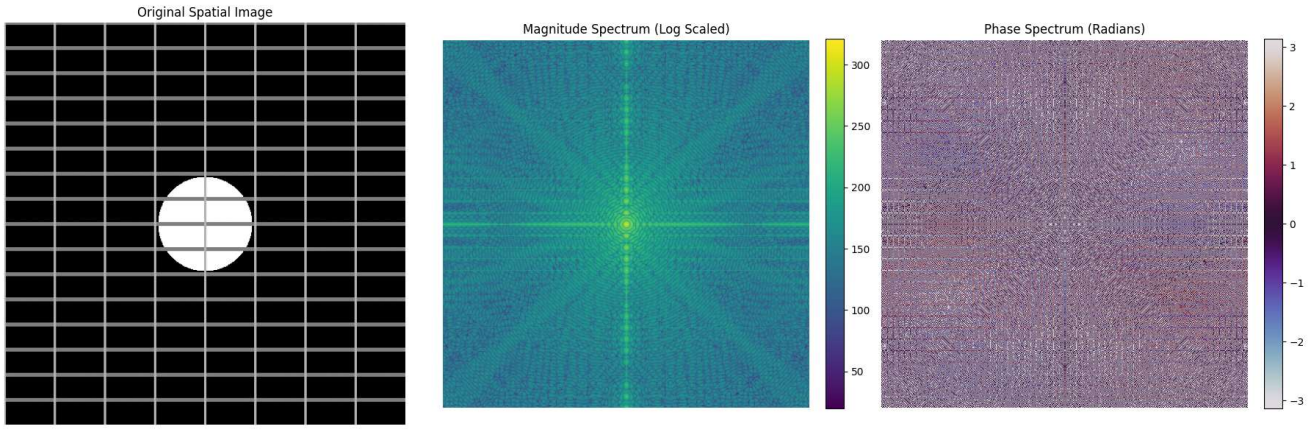
1.Build a tool that computes and visualizes the 2d fourier spectrum (magnitude and phase) of an image

SHREYA(CU24250018)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

def analyze_fourier_2d(image):
    f_transform = np.fft.fft2(image)
    f_shift = np.fft.fftshift(f_transform)
    magnitude_spectrum = np.abs(f_shift)
    magnitude_log = 20 * np.log1p(magnitude_spectrum)
    phase_spectrum = np.angle(f_shift)
    return magnitude_log, phase_spectrum

original_image = generate_synthetic_image()
magnitude, phase = analyze_fourier_2d(original_image)
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(original_image, cmap='gray')
axes[0].set_title('Original Spatial Image')
axes[0].axis('off')
im1 = axes[1].imshow(magnitude, cmap='viridis')
axes[1].set_title('Magnitude Spectrum (Log Scaled)')
axes[1].axis('off')
fig.colorbar(im1, ax=axes[1], fraction=0.046, pad=0.04)
im2 = axes[2].imshow(phase, cmap='twilight')
axes[2].set_title('Phase Spectrum (Radians)')
axes[2].axis('off')
fig.colorbar(im2, ax=axes[2], fraction=0.046, pad=0.04)
plt.tight_layout()
plt.show()
```



2. develop a frequency domain low pass filter (ideal,butterworth,and gaussian) and compare the result with spatial domain edge detection

SHREYA(CU24250018)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

try:
    generate_synthetic_image
```

```
except NameError:
    def generate_synthetic_image(size=(128, 128)):
        img = np.zeros(size)
        cv2.circle(img, (size[1]//2, size[0]//2), 20, 255, -1)
        for i in range(0, size[0], 16):
            cv2.line(img, (0, i), (size[1], i), 127, 1)
        for j in range(0, size[1], 16):
            cv2.line(img, (j, 0), (j, size[0]), 180, 1)
        return img

try:
    mask_butterworth = butterworth_low_pass_filter(image_shape, cutoff_frequency, order=2)
except NameError:
    def butterworth_low_pass_filter(shape, cutoff, order=2):
        rows, cols = shape
        crow, ccol = rows // 2, cols // 2
        y, x = np.ogrid[-crow:rows-crow, -ccol:cols-ccol]
        d = np.sqrt(x*x + y*y)
        mask = 1 / (1 + (d / cutoff)**(2 * order))
        return mask
    mask_butterworth = butterworth_low_pass_filter(image_shape, cutoff_frequency, order=2)

try:
    mask_gaussian = gaussian_low_pass_filter(image_shape, cutoff_frequency)
except NameError:
    def gaussian_low_pass_filter(shape, cutoff):
        rows, cols = shape
        crow, ccol = rows // 2, cols // 2
        y, x = np.ogrid[-crow:rows-crow, -ccol:cols-ccol]
        d2 = x*x + y*y
        mask = np.exp(-d2 / (2 * (cutoff**2)))
        return mask
    mask_gaussian = gaussian_low_pass_filter(image_shape, cutoff_frequency)

try:
    img_ideal = apply_filter_and_reconstruct(original_image, mask_ideal)
except NameError:
    def apply_filter_and_reconstruct(image, mask):
        f_transform = np.fft.fft2(image)
        f_shift = np.fft.fftshift(f_transform)
        filtered_shift = f_shift * mask
        filtered_f = np.fft.ifftshift(filtered_shift)
        reconstructed_image = np.abs(np.fft.ifft2(filtered_f))
        return reconstructed_image
    img_ideal = apply_filter_and_reconstruct(original_image, mask_ideal)

img_butterworth = apply_filter_and_reconstruct(original_image, mask_butterworth)
img_gaussian = apply_filter_and_reconstruct(original_image, mask_gaussian)
```

3. create a frequency domain high pass filter to enhance edges and compare the result with spatial domain edge detection

SHREYA(CU24250O18)

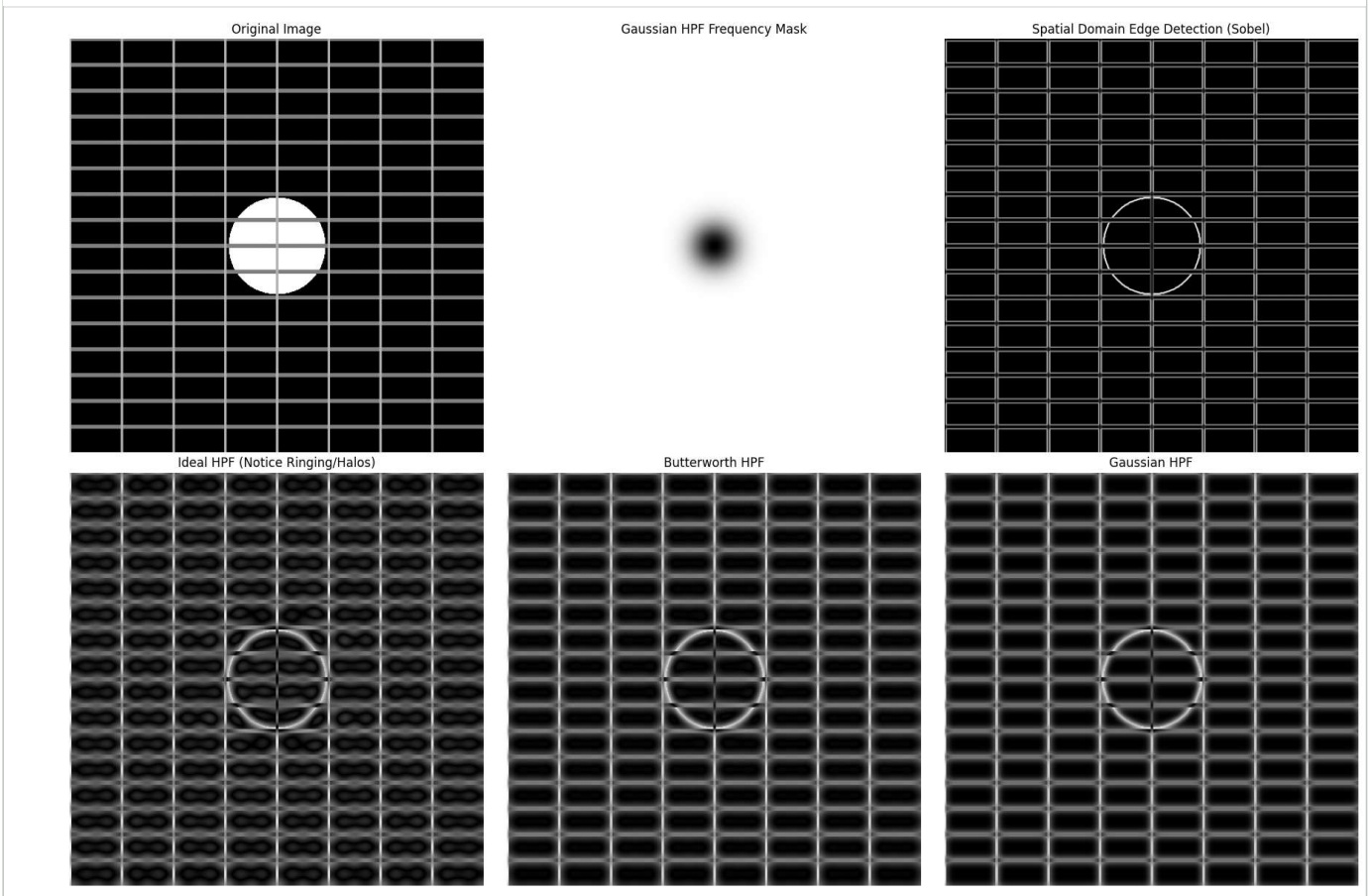
```
def ideal_high_pass_filter(shape, cutoff):
    """Creates an Ideal High Pass Filter mask (1 - Ideal LPF)."""
    return 1 - ideal_low_pass_filter(shape, cutoff)

def butterworth_high_pass_filter(shape, cutoff, order=2):
    """Creates a Butterworth High Pass Filter mask."""
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    y, x = np.ogrid[-crow:rows-crow, -ccol:cols-ccol]
    d = np.sqrt(x*x + y*y)
    # Prevent division by zero
    d[d == 0] = 1e-5
    mask = 1 / (1 + (cutoff / d)**(2 * order))
    return mask

def gaussian_high_pass_filter(shape, cutoff):
    """Creates a Gaussian High Pass Filter mask (1 - Gaussian LPF)."""
    return 1 - gaussian_low_pass_filter(shape, cutoff)
```

```
cutoff_frequency_hpf = 20
image_shape = original_image.shape
mask_ideal_hpf = ideal_high_pass_filter(image_shape, cutoff_frequency_hpf)
```

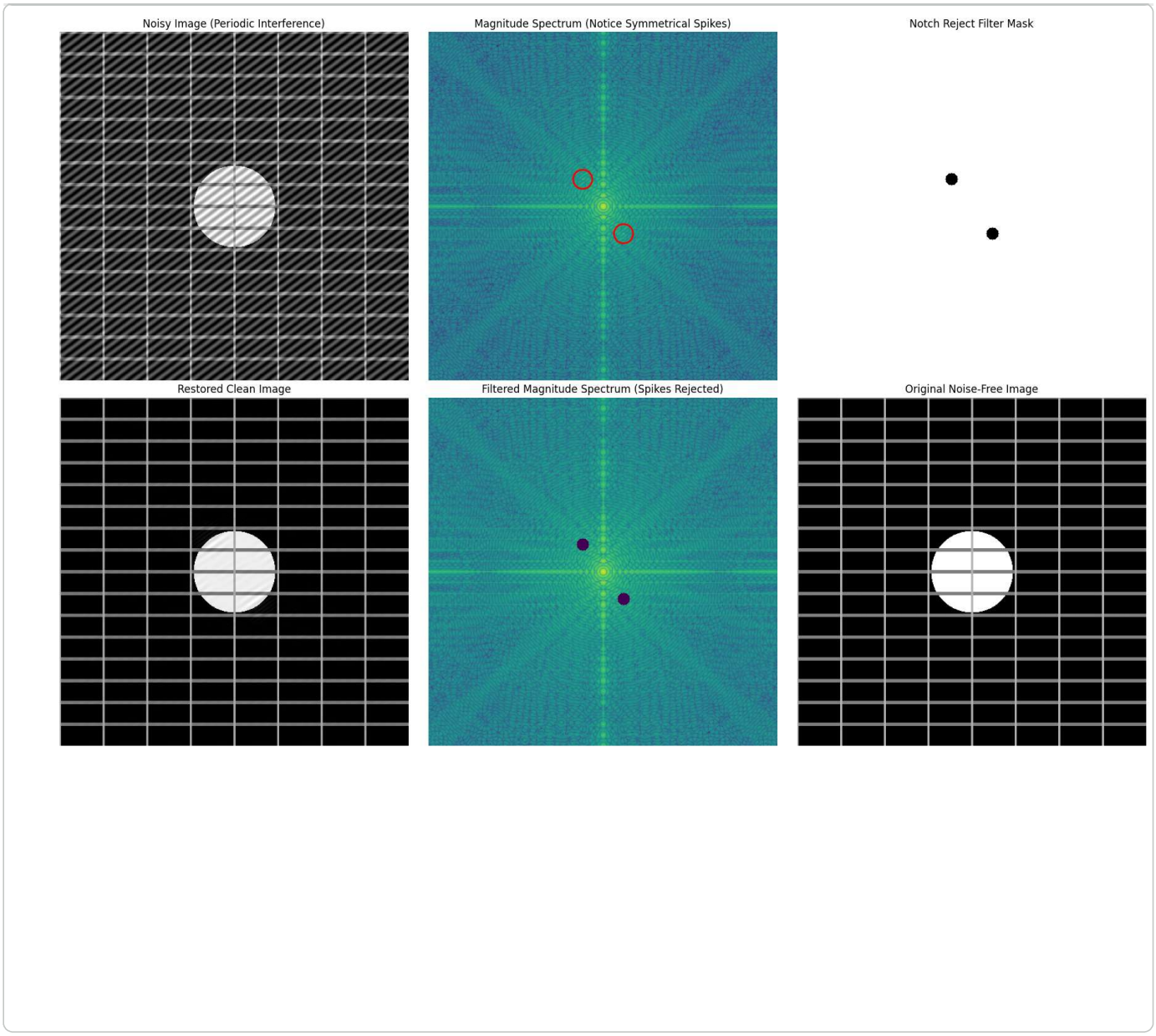
```
mask_butterworth_hpf = butterworth_high_pass_filter(image_shape, cutoff_frequency_hpf, order=2)
mask_gaussian_hpf = gaussian_high_pass_filter(image_shape, cutoff_frequency_hpf)
img_ideal_hpf = apply_filter_and_reconstruct(original_image, mask_ideal_hpf)
img_butterworth_hpf = apply_filter_and_reconstruct(original_image, mask_butterworth_hpf)
img_gaussian_hpf = apply_filter_and_reconstruct(original_image, mask_gaussian_hpf)
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes[0, 0].imshow(original_image, cmap='gray')
axes[0, 0].set_title('Original Image')
axes[0, 0].axis('off')
axes[0, 1].imshow(mask_gaussian_hpf, cmap='gray')
axes[0, 1].set_title('Gaussian HPF Frequency Mask')
axes[0, 1].axis('off')
axes[0, 2].imshow(sobel_magnitude, cmap='gray')
axes[0, 2].set_title('Spatial Domain Edge Detection (Sobel)')
axes[0, 2].axis('off')
axes[1, 0].imshow(img_ideal_hpf, cmap='gray')
axes[1, 0].set_title('Ideal HPF (Notice Ringing/Halos)')
axes[1, 0].axis('off')
axes[1, 1].imshow(img_butterworth_hpf, cmap='gray')
axes[1, 1].set_title('Butterworth HPF')
axes[1, 1].axis('off')
axes[1, 2].imshow(img_gaussian_hpf, cmap='gray')
axes[1, 2].set_title('Gaussian HPF')
axes[1, 2].axis('off')
plt.tight_layout()
plt.show()
```



4.build a periodic noise removal tool that identifies and remove noise spikes visible in the frequency sppectrum

- SHREYA(CU24250018)*

```
import numpy as np
import matplotlib.pyplot as plt
rows, cols = original_image.shape
y, x = np.meshgrid(np.arange(rows), np.arange(cols), indexing='ij')
noise_frequency_y = 40
noise_frequency_x = 30
periodic_noise = 80 * np.sin(2 * np.pi * (noise_frequency_y * y / rows + noise_frequency_x * x / cols))
noisy_image = original_image + periodic_noise
f_transform = np.fft.fft2(noisy_image)
f_shift = np.fft.fftshift(f_transform)
magnitude_noisy = 20 * np.log1p(np.abs(f_shift))
crow, ccol = rows // 2, cols // 2
notch_mask = np.ones((rows, cols))
spike_coords = [
    (crow + noise_frequency_y, ccol + noise_frequency_x),
    (crow - noise_frequency_y, ccol - noise_frequency_x)
]
notch_radius = 9
for r_center, c_center in spike_coords:
    y_idx, x_idx = np.ogrid[-r_center:rows-r_center, -c_center:cols-c_center]
    mask_area = x_idx**2 + y_idx**2 <= notch_radius**2
    notch_mask[mask_area] = 0
filtered_shift = f_shift * notch_mask
magnitude_filtered = 20 * np.log1p(np.abs(filtered_shift))
filtered_f = np.fft.ifftshift(filtered_shift)
reconstructed_clean_image = np.abs(np.fft.ifft2(filtered_f))
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes[0, 0].imshow(noisy_image, cmap='gray')
axes[0, 0].set_title('Noisy Image (Periodic Interference)')
axes[0, 0].axis('off')
im_noisy_mag = axes[0, 1].imshow(magnitude_noisy, cmap='viridis')
axes[0, 1].set_title('Magnitude Spectrum (Notice Symmetrical Spikes)')
axes[0, 1].axis('off')
for r_center, c_center in spike_coords:
    circle = plt.Circle((c_center, r_center), notch_radius + 5, color='red', fill=False, linewidth=2)
    axes[0, 1].add_patch(circle)
axes[0, 2].imshow(notch_mask, cmap='gray')
axes[0, 2].set_title('Notch Reject Filter Mask')
axes[0, 2].axis('off')
axes[1, 0].imshow(reconstructed_clean_image, cmap='gray')
axes[1, 0].set_title('Restored Clean Image')
axes[1, 0].axis('off')
axes[1, 1].imshow(magnitude_filtered, cmap='viridis')
axes[1, 1].set_title('Filtered Magnitude Spectrum (Spikes Rejected)')
axes[1, 1].axis('off')
axes[1, 2].imshow(original_image, cmap='gray')
axes[1, 2].set_title('Original Noise-Free Image')
axes[1, 2].axis('off')
plt.tight_layout()
plt.show()
```

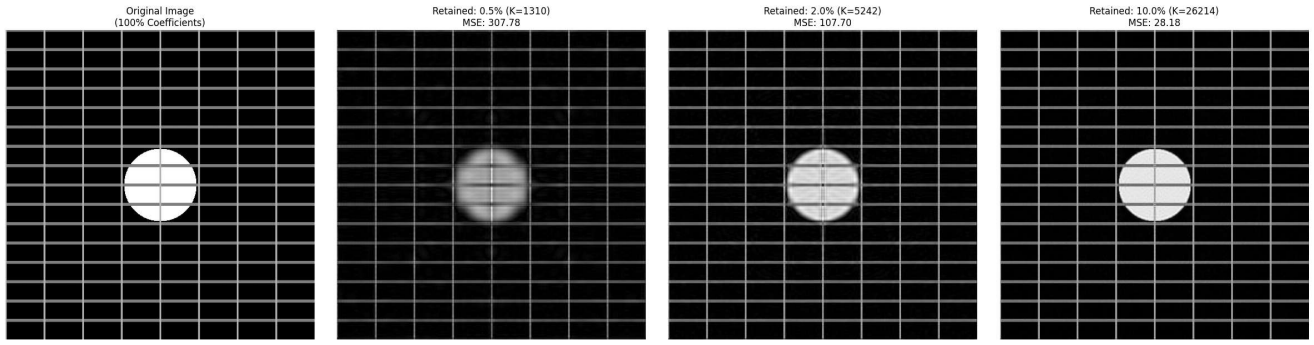


5.develop a program that compresses b retaining only the tp K fourier coefficients and reconstructs it

SHREYA(CU24250018)

```
import numpy as np
import matplotlib.pyplot as plt
def compress_fourier_top_k(image, keep_fraction=0.05):
    f_transform = np.fft.fft2(image)
    magnitudes = np.abs(f_transform)
    flat_magnitudes = magnitudes.flatten()
    total_coefficients = flat_magnitudes.size
    k = int(total_coefficients * keep_fraction)
    threshold_val = np.partition(flat_magnitudes, -k)[-k]
    compressed_f = f_transform.copy()
    compressed_f[magnitudes < threshold_val] = 0
    reconstructed_image = np.abs(np.fft.ifft2(compressed_f))
    mse = np.mean((image - reconstructed_image) ** 2)
    return reconstructed_image, k, mse
fractions_to_test = [0.005, 0.02, 0.10]
results = []
for frac in fractions_to_test:
    recon_img, retained_k, mse_val = compress_fourier_top_k(original_image, keep_fraction=frac)
    results.append((recon_img, retained_k, mse_val, frac))
fig, axes = plt.subplots(1, 4, figsize=(24, 6))
axes[0].imshow(original_image, cmap='gray')
axes[0].set_title('Original Image\n(100% Coefficients)')
axes[0].axis('off')
```

```
for i, (recon_img, k_val, mse_val, frac) in enumerate(results):
    percentage = frac * 100
    axes[i+1].imshow(recon_img, cmap='gray')
    axes[i+1].set_title(f'Retained: {percentage}% (K={k_val})\nMSE: {mse_val:.2f}')
    axes[i+1].axis('off')
plt.tight_layout()
plt.show()
```



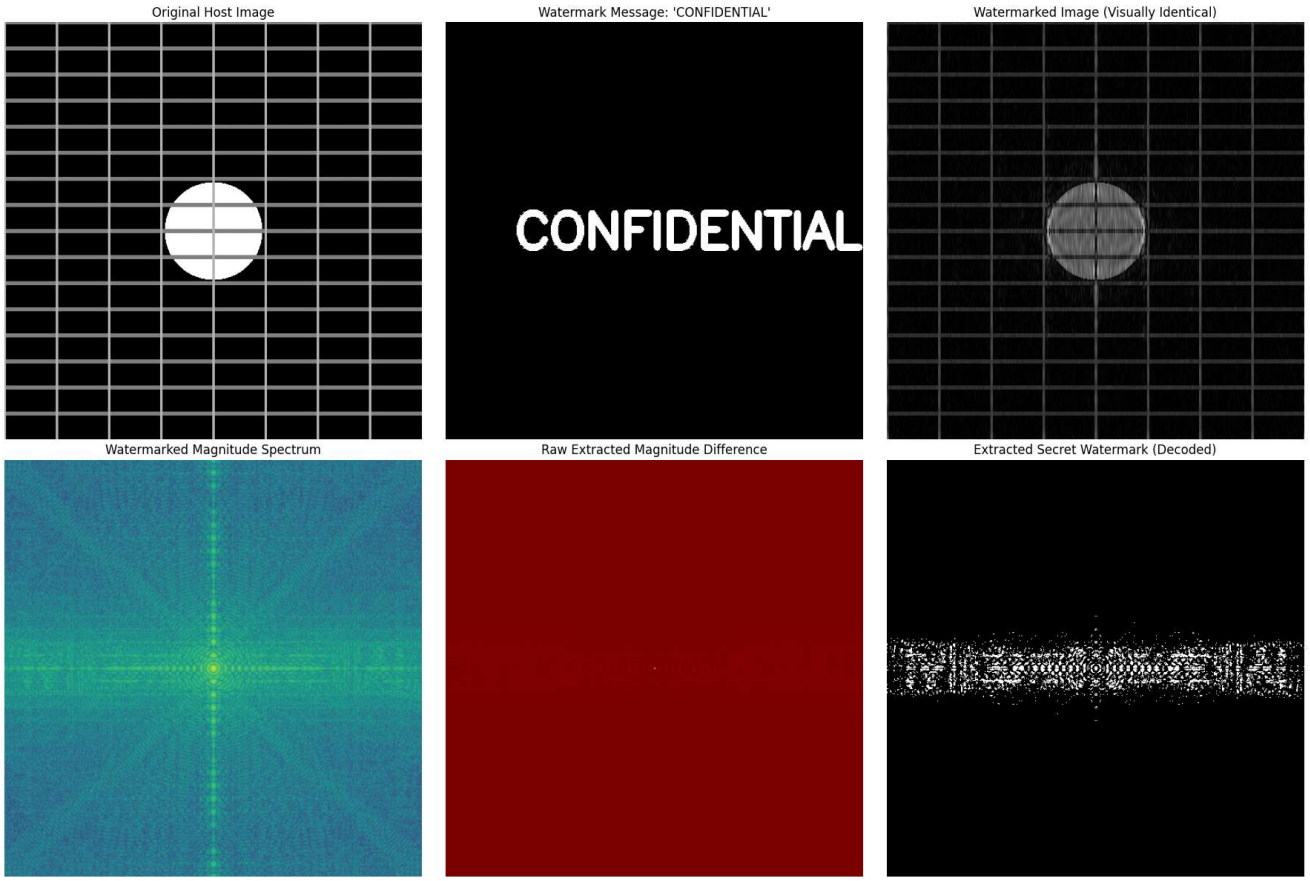
6.create a watermark hide reveal tool that embeds and extracts a watermarrk using frequency domain manipulation

SHREYA(CU24250018)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

def embed_watermark(host_image, watermark_text, alpha=5.0):
    h, w = host_image.shape
    watermark_mask = np.zeros((h, w), dtype=np.uint8)
    cv2.putText(watermark_mask, watermark_text, (w//6, h//2 + 20),
                cv2.FONT_HERSHEY_SIMPLEX, 2.0, 255, 5, cv2.LINE_AA)
    watermark_bin = (watermark_mask > 0).astype(float)
    f_transform = np.fft.fft2(host_image)
    f_shift = np.fft.fftshift(f_transform)
    magnitude = np.abs(f_shift)
    phase = np.angle(f_shift)
    watermark_symmetric = watermark_bin + np.flip(watermark_bin)
    watermark_symmetric = (watermark_symmetric > 0).astype(float)
    watermarked_magnitude = magnitude + alpha * watermark_symmetric * (magnitude.mean())
    watermarked_shift = watermarked_magnitude * np.exp(1j * phase)
    watermarked_f = np.fft.ifftshift(watermarked_shift)
    watermarked_image = np.abs(np.fft.ifft2(watermarked_f))
    return watermarked_image, watermark_bin
def extract_watermark(watermarked_image, original_image, alpha=5.0):
    f_watermarked = np.fft.fftshift(np.fft.fft2(watermarked_image))
    f_original = np.fft.fftshift(np.fft.fft2(original_image))
    mag_watermarked = np.abs(f_watermarked)
    mag_original = np.abs(f_original)
    difference = mag_watermarked - mag_original
    extracted = difference / (alpha * mag_original.mean() + 1e-9)
    extracted_binary = extracted > 0.5
    return extracted, extracted_binary
secret_message = "CONFIDENTIAL"
watermarked_img, original_watermark = embed_watermark(original_image, secret_message, alpha=8.0)
extracted_raw, extracted_bin = extract_watermark(watermarked_img, original_image, alpha=8.0)
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes[0, 0].imshow(original_image, cmap='gray')
axes[0, 0].set_title('Original Host Image')
axes[0, 0].axis('off')
axes[0, 1].imshow(original_watermark, cmap='gray')
axes[0, 1].set_title(f"Watermark Message: '{secret_message}'")
axes[0, 1].axis('off')
axes[0, 2].imshow(watermarked_img, cmap='gray')
axes[0, 2].set_title('Watermarked Image (Visually Identical)')
axes[0, 2].axis('off')
f_shift_wm = np.fft.fftshift(np.fft.fft2(watermarked_img))
```

```
mag_wm_log = 20 * np.log1p(np.abs(f_shift_wm))
axes[1, 0].imshow(mag_wm_log, cmap='viridis')
axes[1, 0].set_title('Watermarked Magnitude Spectrum')
axes[1, 0].axis('off')
axes[1, 1].imshow(extracted_raw, cmap='hot')
axes[1, 1].set_title('Raw Extracted Magnitude Difference')
axes[1, 1].axis('off')
axes[1, 2].imshow(extracted_bin, cmap='gray')
axes[1, 2].set_title('Extracted Secret Watermark (Decoded)')
axes[1, 2].axis('off')
plt.tight_layout()
plt.show()
```



7. build a band pass / band reject filter designer for isolating specific frequency ranges in a textured image

SHREYA(CU24250018)

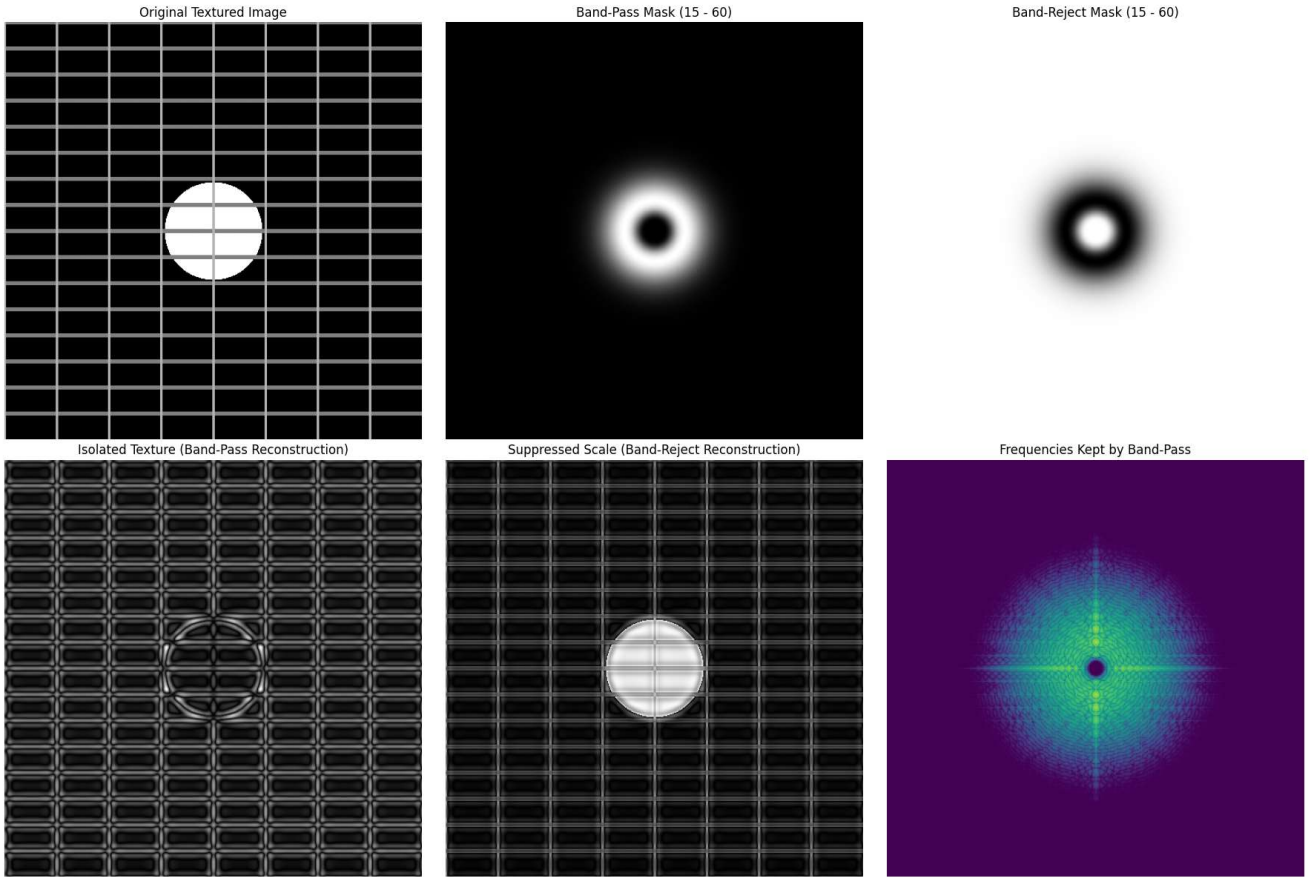
```
import numpy as np
import matplotlib.pyplot as plt

def band_filters_designer(shape, cutoff_inner, cutoff_outer, filter_type='gaussian'):
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    y, x = np.ogrid[-crow:rows-crow, -ccol:cols-ccol]
    d2 = x*x + y*y
    d = np.sqrt(d2)
    if filter_type == 'ideal':
```

```
            br_mask = np.ones((rows, cols))
            band_area = (d >= cutoff_inner) & (d <= cutoff_outer)
            br_mask[band_area] = 0
            bp_mask = 1 - br_mask
        else:
            w = cutoff_outer - cutoff_inner  # Bandwidth
            d0 = (cutoff_outer + cutoff_inner) / 2.0
            d[d == 0] = 1e-5
            br_mask = 1 - np.exp(-((d2 - d0**2) / (d * w + 1e-5))**2)
            bp_mask = 1 - br_mask
        return bp_mask, br_mask

axes[1, 1].imshow(img_band_reject, cmap='gray')
axes[1, 1].set_title('Suppressed Scale (Band-Reject Reconstruction)')
axes[1, 1].axis('off')
f_shift_orig = np.fft.fftshift(np.fft.fft2(original_image))
active_frequencies = np.abs(f_shift_orig) * bp_mask
active_freq_log = 20 * np.log1p(active_frequencies)
axes[1, 2].imshow(active_freq_log, cmap='viridis')
axes[1, 2].set_title('Frequencies Kept by Band-Pass')
axes[1, 2].axis('off')

plt.tight_layout()
plt.show()
```



8.develop a demo that swaps the phase and magnitude components between two imagesto show their relative importance

SHREYA(CU24250018)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

size = (128, 128)
image_a = np.zeros(size)
cv2.rectangle(image_a, (20, 20), (108, 108), 255, -1)
cv2.circle(image_a, (64, 64), 25, 0, -1)

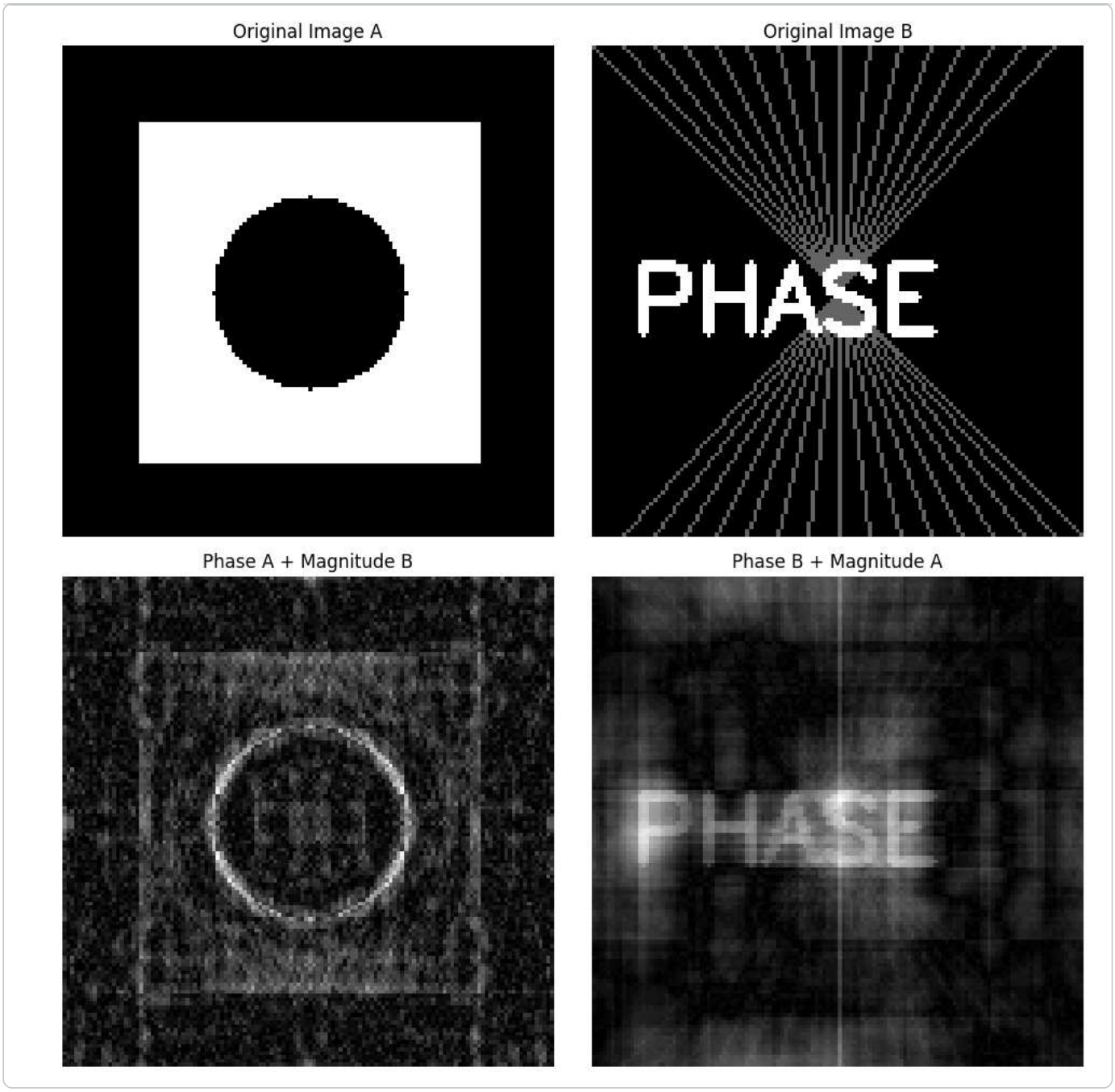
image_b = np.zeros(size)
for i in range(0, 128, 8):
    cv2.line(image_b, (i, 0), (128 - i, 128), 100, 1)
cv2.putText(image_b, "PHASE", (10, 74), cv2.FONT_HERSHEY_SIMPLEX, 0.8, 255, 2, cv2.LINE_AA)

f_a = np.fft.fft2(image_a)
f_b = np.fft.fft2(image_b)

mag_a, phase_a = np.abs(f_a), np.angle(f_a)
mag_b, phase_b = np.abs(f_b), np.angle(f_b)

reconstructed_1 = mag_b * np.exp(1j * phase_a)
img_recon_1 = np.abs(np.fft.ifft2(reconstructed_1))

reconstructed_2 = mag_a * np.exp(1j * phase_b)
img_recon_2 = np.abs(np.fft.ifft2(reconstructed_2))
```



9.create a blur detection tool that flags blurry image based on the high frequency energy content of their spectrum

SHREYA(CU24250018)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

def detect_blur_fft(image, low_freq_radius=15, threshold=10.0):
    f_transform = np.fft.fft2(image)
    f_shift = np.fft.fftshift(f_transform)
    magnitude = np.abs(f_shift)
    h, w = image.shape
    crow, ccol = h // 2, w // 2
    y, x = np.ogrid[-crow:h-crow, -ccol:w-ccol]
    low_freq_mask = x*x + y*y <= low_freq_radius*low_freq_radius
    high_freq_magnitude = magnitude.copy()
    high_freq_magnitude[low_freq_mask] = 0
    high_freq_energy = np.mean(20 * np.log1p(high_freq_magnitude))
    is_blurry = high_freq_energy < threshold
    return high_freq_energy, is_blurry, high_freq_magnitude

# Downscale the input image to 128x128
```

```
sharp_img = cv2.resize(original_image, (128, 128)).astype(np.uint8)
blurry_img = cv2.GaussianBlur(sharp_img, (5, 5), 0)

energy_threshold = 12.0
sharp_energy, is_sharp_blurry, sharp_high_freq = detect_blur_fft(sharp_img, threshold=energy_threshold)
blurry_energy, is_blurry_blurry, blurry_high_freq = detect_blur_fft(blurry_img, threshold=energy_threshold)

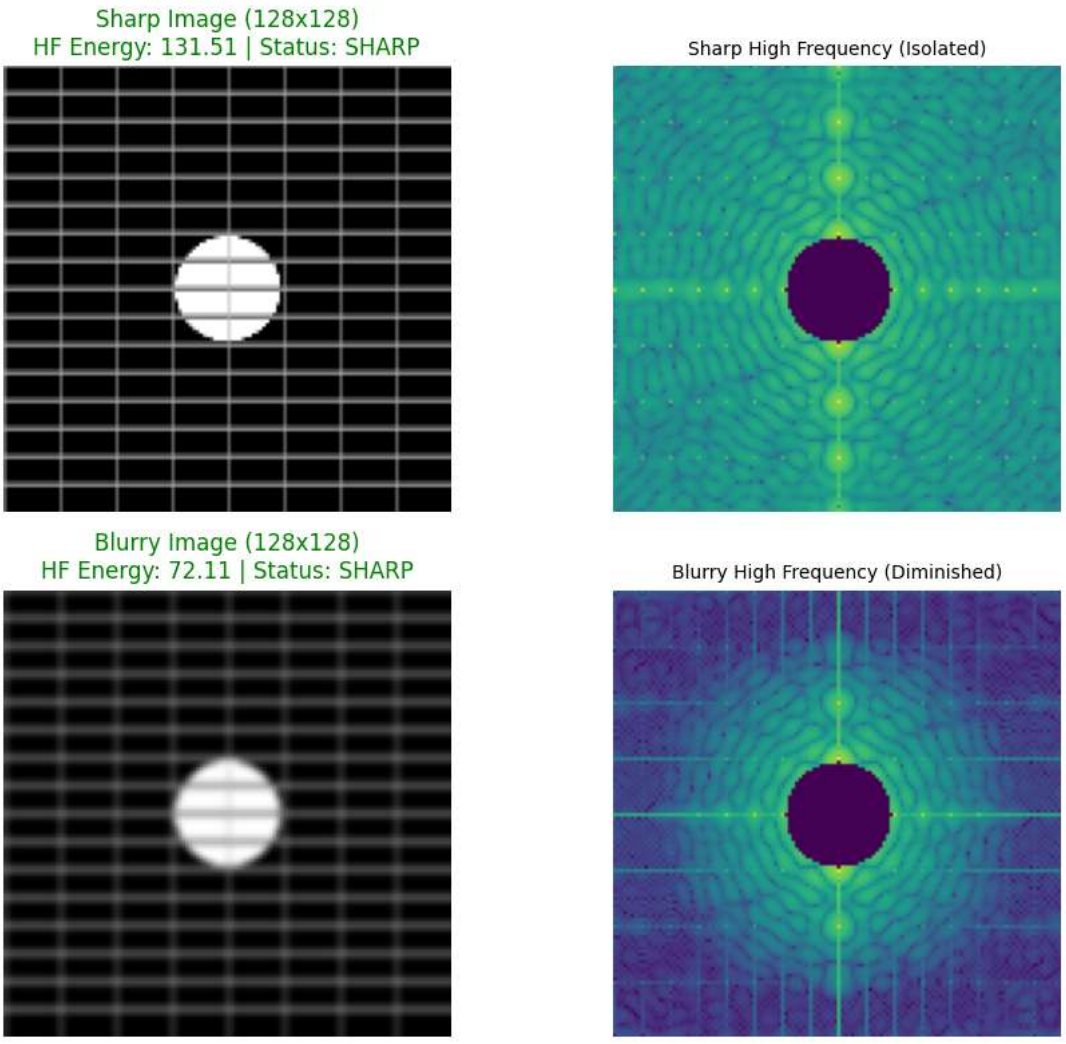
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].imshow(sharp_img, cmap='gray')
axes[0, 0].set_title(f"Sharp Image (128x128)\nHF Energy: {sharp_energy:.2f} | Status: {'BLURRY' if is_sharp_blurry else 'SHARP'}
                     color='green' if not is_sharp_blurry else 'red', fontsize=12)
axes[0, 0].axis('off')

axes[0, 1].imshow(20 * np.log1p(sharp_high_freq), cmap='viridis')
axes[0, 1].set_title("Sharp High Frequency (Isolated)", fontsize=10)
axes[0, 1].axis('off')

axes[1, 0].imshow(blurry_img, cmap='gray')
axes[1, 0].set_title(f"Blurry Image (128x128)\nHF Energy: {blurry_energy:.2f} | Status: {'BLURRY' if is_blurry_blurry else 'SHARP
                     color='red' if is_blurry_blurry else 'green', fontsize=12)
axes[1, 0].axis('off')

axes[1, 1].imshow(20 * np.log1p(blurry_high_freq), cmap='viridis')
axes[1, 1].set_title("Blurry High Frequency (Diminished)", fontsize=10)
axes[1, 1].axis('off')

plt.tight_layout()
plt.show()
```



10. build a side by side visualizer comparing spatial domain filtering resuts for the same image

SHREYA(CU24250O18)

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

# Ensure base_image is small (128x128)
base_image = cv2.resize(original_image, (128, 128)).astype(np.uint8)

mean_blur = cv2.blur(base_image, (9, 9))
gaussian_blur = cv2.GaussianBlur(base_image, (9, 9), 2.0)

noisy_spatial = base_image.copy()
noise_mask_salt = np.random.rand(*base_image.shape) < 0.02
noise_mask_pepper = np.random.rand(*base_image.shape) < 0.02
noisy_spatial[noise_mask_salt] = 255
noisy_spatial[noise_mask_pepper] = 0
median_filtered = cv2.medianBlur(noisy_spatial, 5)

sobel_x_comp = cv2.Sobel(base_image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y_comp = cv2.Sobel(base_image, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = np.sqrt(sobel_x_comp**2 + sobel_y_comp**2)
laplacian_edges = np.abs(cv2.Laplacian(base_image, cv2.CV_64F))

ig, axes = plt.subplots(2, 3, figsize=(12, 8))
axes[0, 0].imshow(base_image, cmap='gray')
axes[0, 0].set_title('Original Image (Small)')
axes[0, 0].axis('off')

axes[0, 1].imshow(mean_blur, cmap='gray')
axes[0, 1].set_title('Mean (Box) Filter (9x9)')
axes[0, 1].axis('off')

axes[0, 2].imshow(gaussian_blur, cmap='gray')
axes[0, 2].set_title('Gaussian Filter (9x9, sigma=2)')
axes[0, 2].axis('off')

axes[1, 0].imshow(noisy_spatial, cmap='gray')
axes[1, 0].set_title('S&P Noisy -> Median Filtered (Right)')
axes[1, 0].axis('off')
```